```
    change_table :users do |t|
      t.boolean :admin, default: false
    end
  end
end
```

Let's think about where this needs to be tested. The behavior of the User#can_view? method needs to change. Crucially, though, the controller code won't change; it will still just call can_view?. The logic is still *if users can view, let them; redirect otherwise*. So you don't need to write new integration tests. You can write this logic as unit tests:

security/03/spec/models/user_spec.rb

```
describe "public roles" do
  it "allows an admin to view a project" do
    user.admin = true
    expect(user.can_view?(project)).to be_truthy
  end

  it "allows an user to view a public project" do
    project.public = true
    expect(user.can_view?(project)).to be_truthy
  end
end
```

These tests set up the pass-through condition for administrators and public projects—the negative condition is already covered by the "user cannot view an unrelated project" test.

And the test passes with an additional line in the method:

security/03/app/models/user.rb

```
def can_view?(project)
  return true if admin || project.public?
  project.in?(projects)
end
```

Now we're at a refactoring step, but we seem pretty clean at the moment.

There's an open question in the testing strategy: whether you should have started the process of adding admin users and public projects with an end-to-end integration test.

The answer to the question depends on the goal of your tests.

From a TDD integration-test perspective, you don't need an integration test because the integration logic didn't change. The code changes you made were localized to a single class, so the behavior of any code that uses that class is unchanged. Although you could write an integration test that would expose